

Terraform

Hands-On Learning Notes

A beginner-friendly, practical walkthrough of Terraform — infrastructure as code for AWS — documented step-by-step while practicing in VS Code and the AWS Console.

Prepared by Lasya Mandapati

Topics Covered

count & for_each — creating multiple resources from one block

parallelism — controlling how many resources Terraform builds at once

depends_on & prevent_destroy — controlling order and protecting resources

ignore_changes & create_before_destroy — lifecycle rules

terraform import, taint/replace, and refresh — working with existing infrastructure

terraform debugging & terraform graph — troubleshooting and visualizing

Remote state with an S3 backend

local blocks & variables — writing reusable, flexible configuration

terraform workspaces — managing dev/test/prod environments

The Terraform CLI & dynamic blocks

Provisioners — local-exec and remote-exec

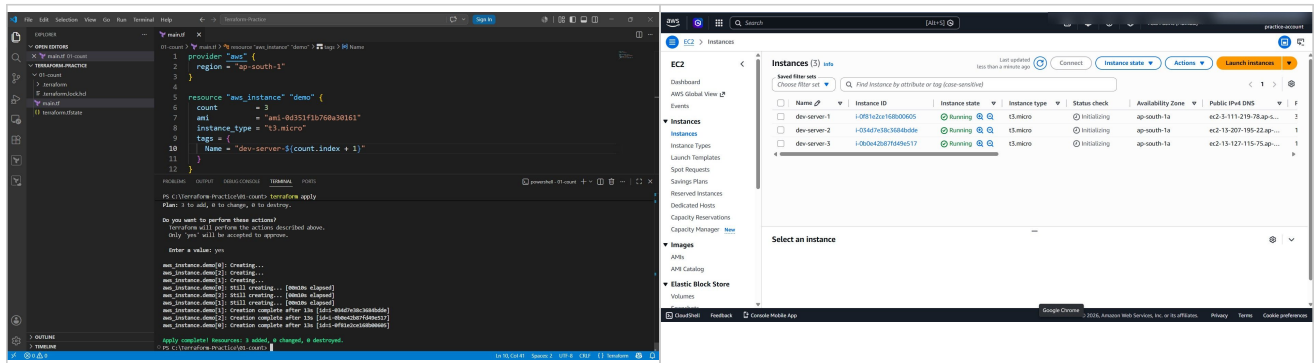
Final project — a 3-tier AWS architecture built entirely with Terraform

Creating Multiple Resources

What's the problem? Writing a separate block of code for every single server you want is repetitive. If you need 3 identical servers, copy-pasting the same resource block three times is messy and hard to maintain. Terraform gives you a few different tools to avoid that repetition.

01 count — repeat a resource a fixed number of times

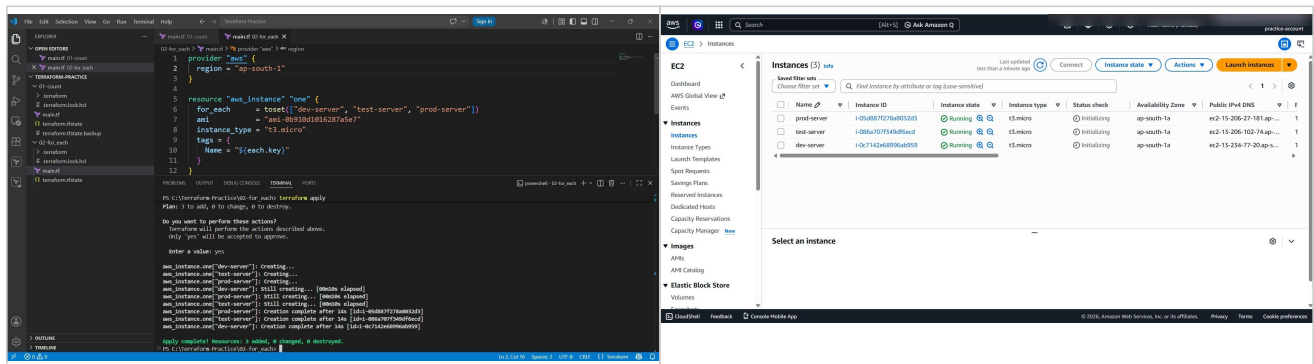
Adding `count = 3` inside a resource block tells Terraform to create three copies of it. Each copy can be told apart using `count.index` (which starts at 0), so each server gets a unique name like `dev-server-1`, `dev-server-2`, and `dev-server-3`.



Left: the Terraform code using `count = 3` and `count.index` inside the tag name, plus the terminal confirming all three servers were created. Right: all three “dev-server” instances now visible and running in the EC2 console.

02 for_each — repeat a resource using named values

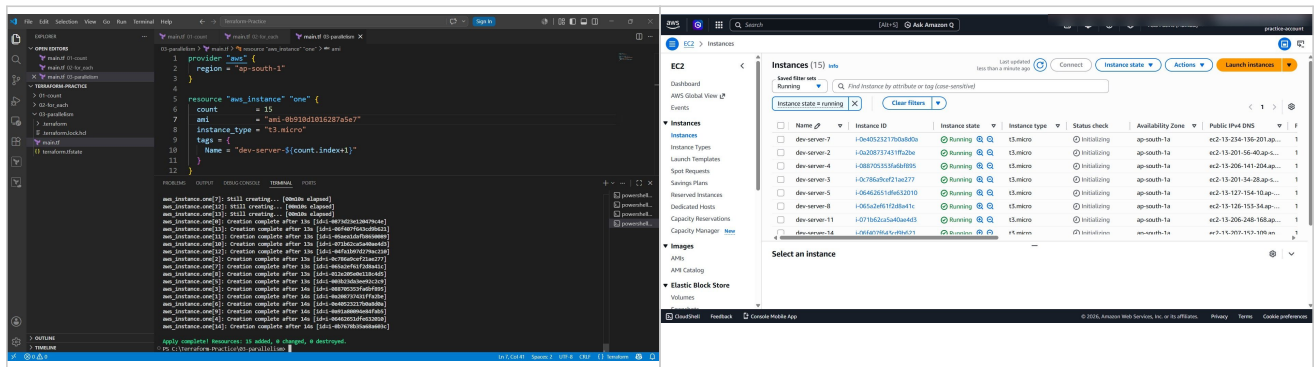
`for_each` does a similar job to `count`, but instead of just numbering copies 0, 1, 2, it lets you loop over a list of meaningful names (like “dev-server”, “test-server”, “prod-server”) and give each resulting server that exact name. This is usually clearer than `count` when each copy genuinely represents something different.



Left: `for_each` looping over a set of three named servers. Right: the three resulting instances in EC2, each correctly named after the value it was created from.

03 parallelism — controlling how fast Terraform builds things

By default, Terraform tries to create up to 10 resources at the same time to save time. The `-parallelism=N` flag lets you change that number — useful when you want to slow things down (e.g. to avoid hitting an AWS API rate limit) or see resources being created one at a time for learning purposes.

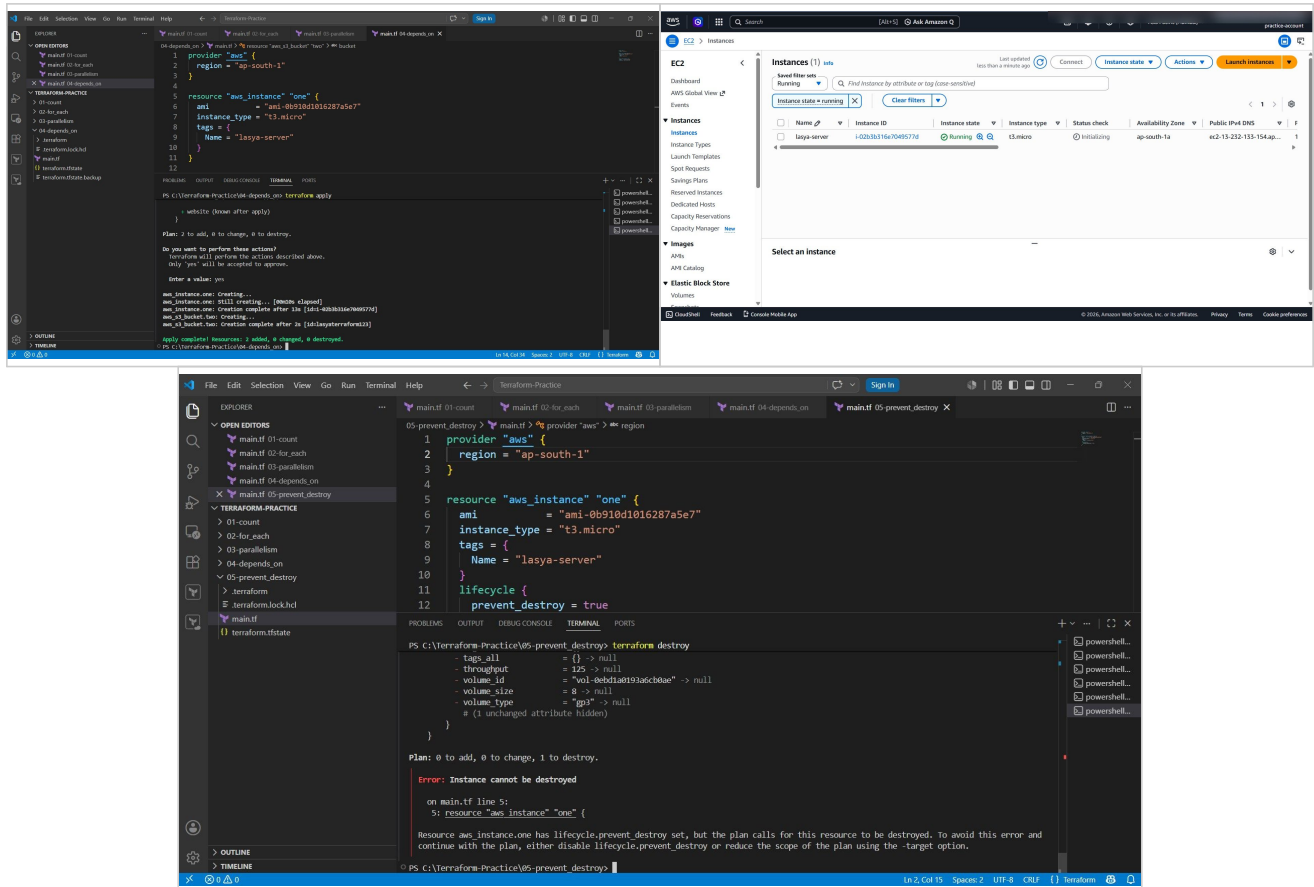


Left: several instances being created with a controlled parallelism setting, visible in the terminal output as they complete one after another. Right: all the resulting servers, now running in EC2.

Controlling Order & Protecting Resources

04 depends_on — forcing one resource to wait for another

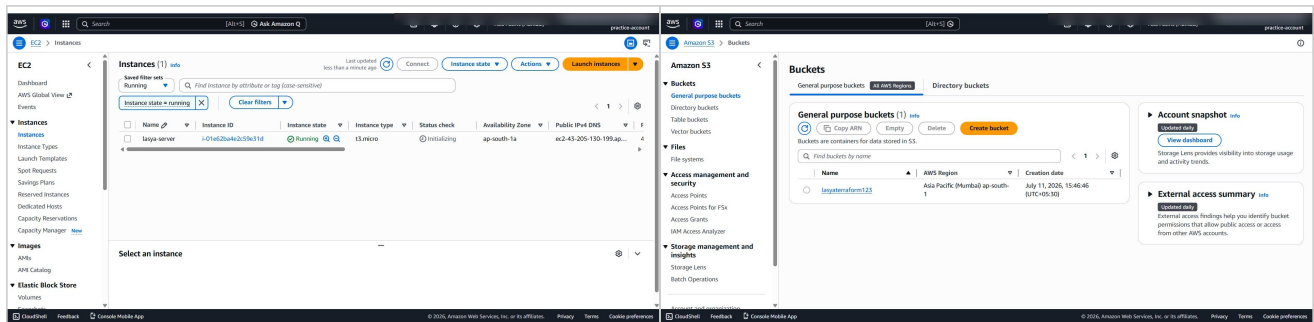
Terraform usually figures out the right order to create resources on its own, based on references between them. But sometimes two resources aren't directly linked in code even though one genuinely needs to exist first. `depends_on` lets you explicitly say “wait for this other resource before creating me.”



The EC2 instance is created only after the resource it depends on is ready, confirmed by the terminal output and the instance state in the console. The S3 bucket shown below is the related resource being tracked as part of the same setup.

05 prevent_destroy — stopping accidental deletion

Some resources are too important to risk deleting by accident — a production database, for example. Adding `prevent_destroy = true` inside a resource's lifecycle block makes Terraform refuse to destroy that resource, even if a terraform destroy command is run. You'd have to deliberately remove that protection first.



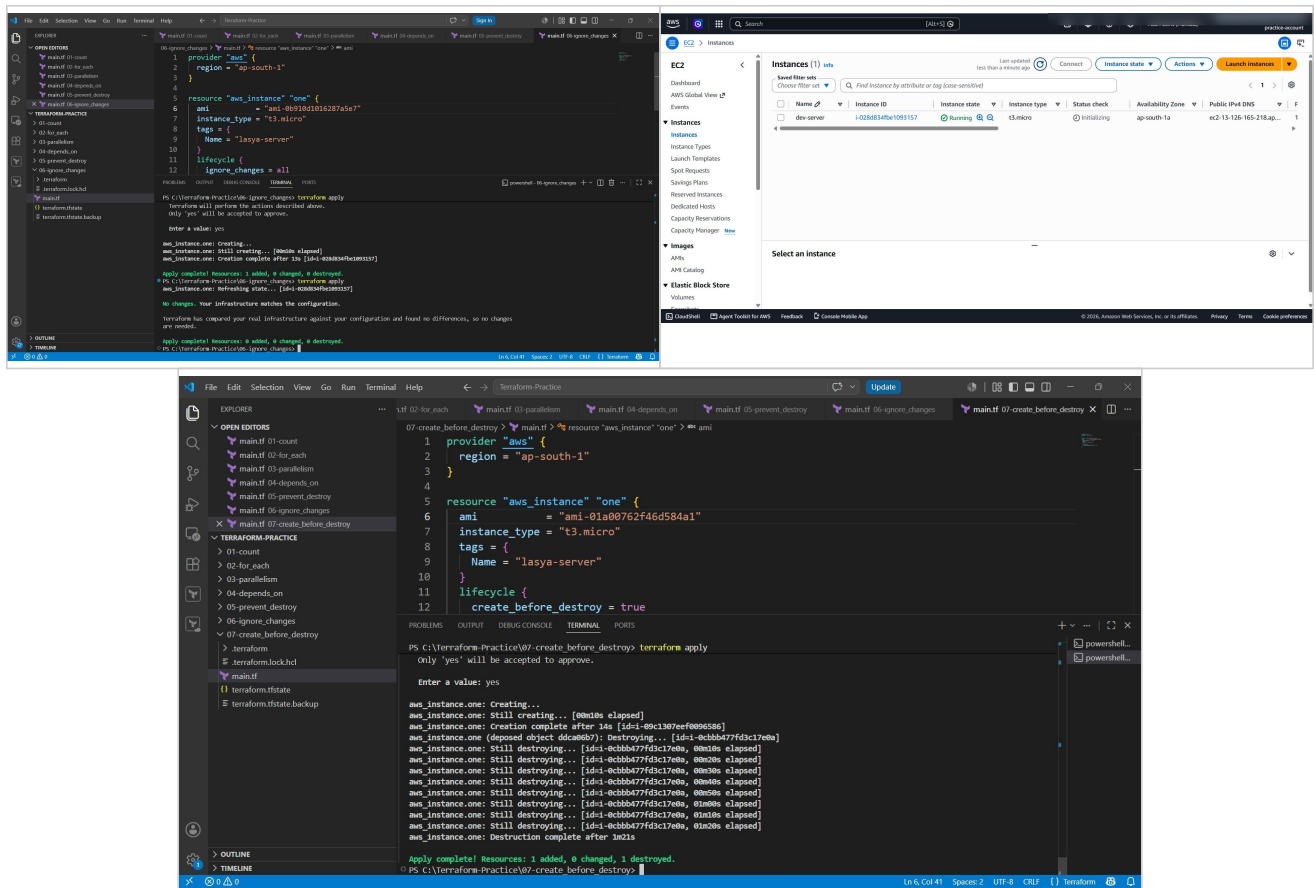
Left: the protected resource's configuration and the terminal refusing to destroy it. Right: the instance still running safely in the console, untouched by the destroy attempt.

Lifecycle Rules

Terraform constantly compares your code to what's actually running, and tries to make reality match the code. **Lifecycle rules** let you fine-tune that behavior for specific situations.

06 ignore_changes — letting some changes slide

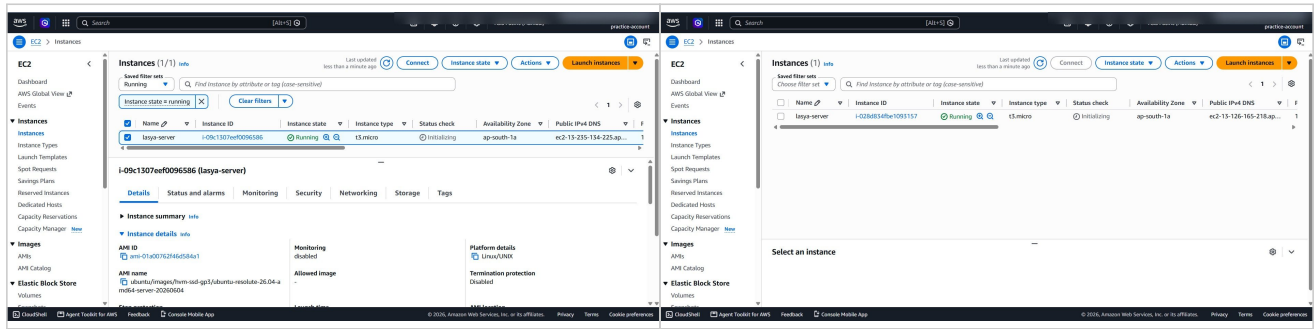
If something about a resource is expected to change outside of Terraform (for example, tags added manually in the AWS Console), `ignore_changes` tells Terraform not to treat that as configuration drift it needs to fix. Without this, Terraform would try to revert the manual change on every apply.



Left: the lifecycle block configured with `ignore_changes`. Right and below: the instance running normally in the console, unaffected by the ignored attribute.

07 create_before_destroy — avoiding downtime on replacement

Normally, if a resource needs to be replaced, Terraform destroys the old one first and then creates the new one — which briefly means no server is running at all. `create_before_destroy = true` flips that order: the new resource is created and ready *before* the old one is removed, avoiding a gap in availability.

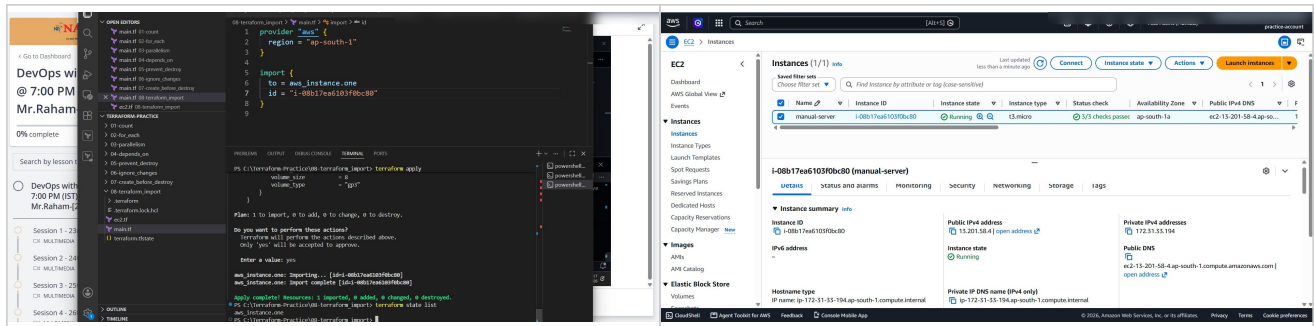


Left: the lifecycle rule in the Terraform code. Right: the replacement instance's details, including its AMI and termination protection settings, confirming the new server was stood up cleanly.

Working With Existing Infrastructure

08 terraform import — bringing existing resources under Terraform's control

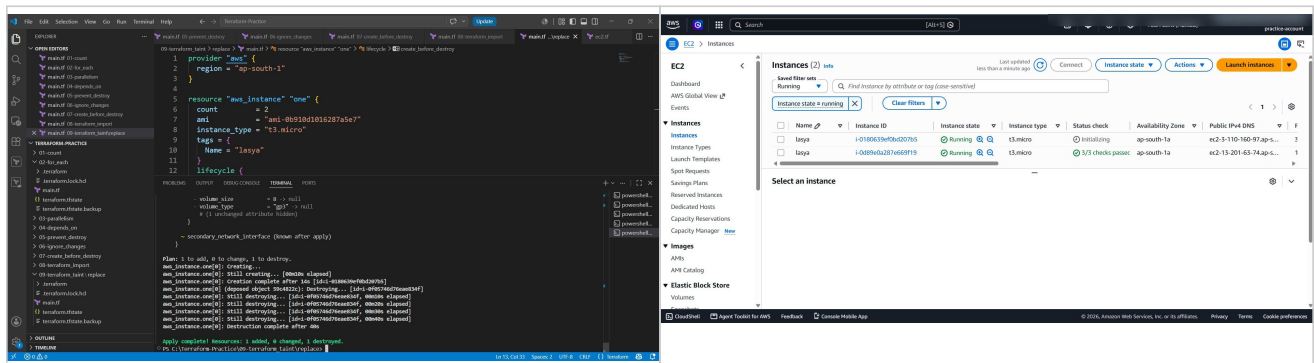
Sometimes a server was created manually (through the AWS Console, not Terraform) and you want Terraform to start managing it going forward. terraform import links an existing real-world resource to a resource block in your code, without recreating it — Terraform simply starts tracking something that already exists.



Left: running terraform import to link an existing instance ID to a resource block named "one" in the code. Right: that same instance ("manual-server") now visible in the console, exactly as it was before — only now Terraform knows about it too.

09 terraform taint / replace — forcing a resource to be rebuilt

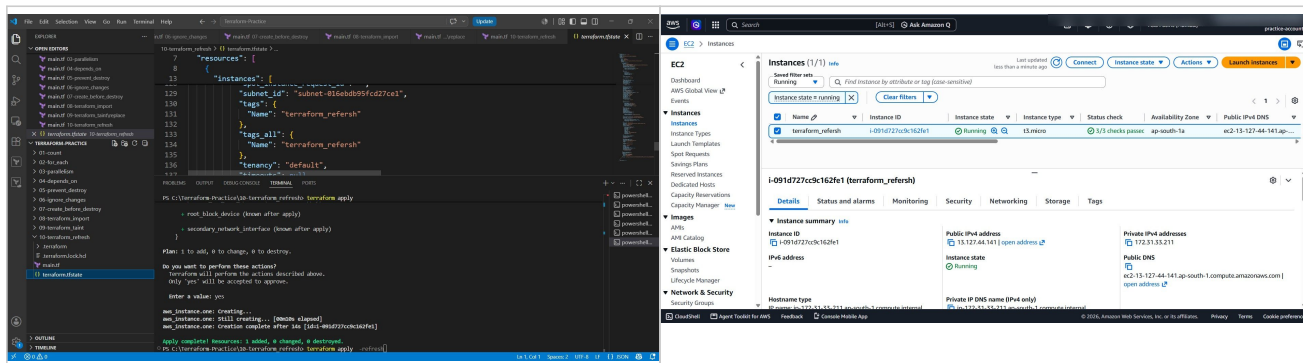
If a server ends up in a broken or unwanted state but Terraform doesn't detect any configuration drift, you can manually mark it for replacement. The newer terraform apply -replace=<resource> command (which replaced the older terraform taint) destroys and recreates that one specific resource on the next apply, without touching anything else.



Left: the resource configuration and lifecycle settings for the server being replaced. Right: the freshly recreated instance, now running with a new instance ID.

10 terraform refresh — syncing Terraform's records with reality

terraform refresh updates Terraform's saved state file to match the real, current state of your infrastructure — without changing any actual resources. This is useful if something changed outside of Terraform and you just want Terraform's records to catch up.

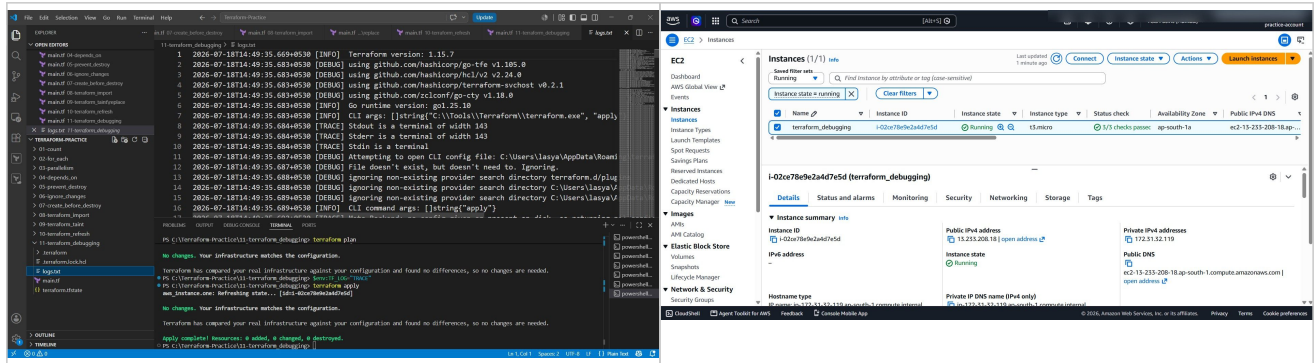


Left: the Terraform configuration with tags, and the refresh command being run. Right: the “terraform_refresh” instance details in the console, matching what Terraform's state now reflects.

Troubleshooting & Visualizing

11 terraform debugging — seeing exactly what Terraform is doing

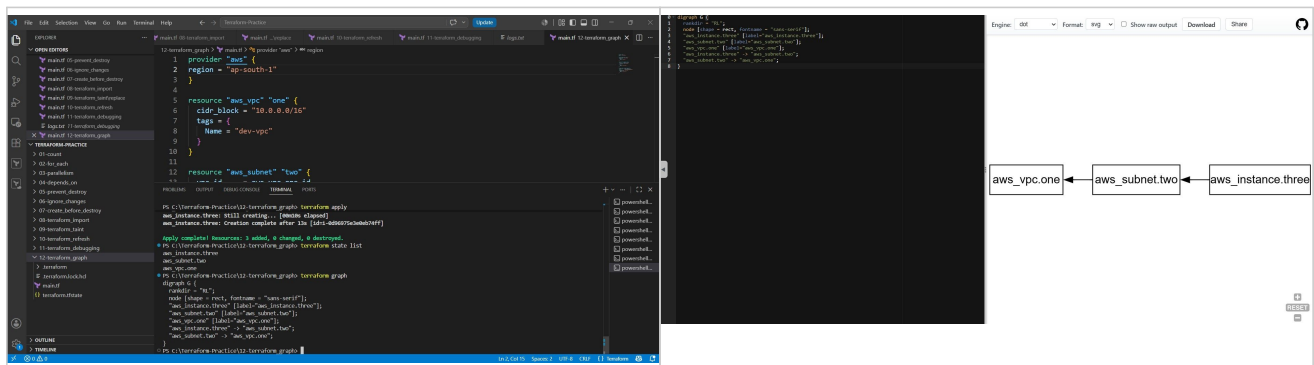
When something isn't working as expected, Terraform can be set to print much more detailed logs — showing every internal step, file it reads, and decision it makes. This is usually turned on by setting an environment variable such as `TF_LOG=TRACE` before running a command, which is exactly what's happening in the terminal output below.



Left: verbose debug-level logging showing Terraform's internal steps, from checking plugin versions to reading configuration files.
Right: the resulting "terraform_debugging" instance running successfully in the console.

12 terraform graph — visualizing how resources connect

terraform graph generates a diagram showing how all the resources in your configuration depend on and connect to each other. This is especially useful for understanding a large, complex setup at a glance, or for spotting an unintended dependency.

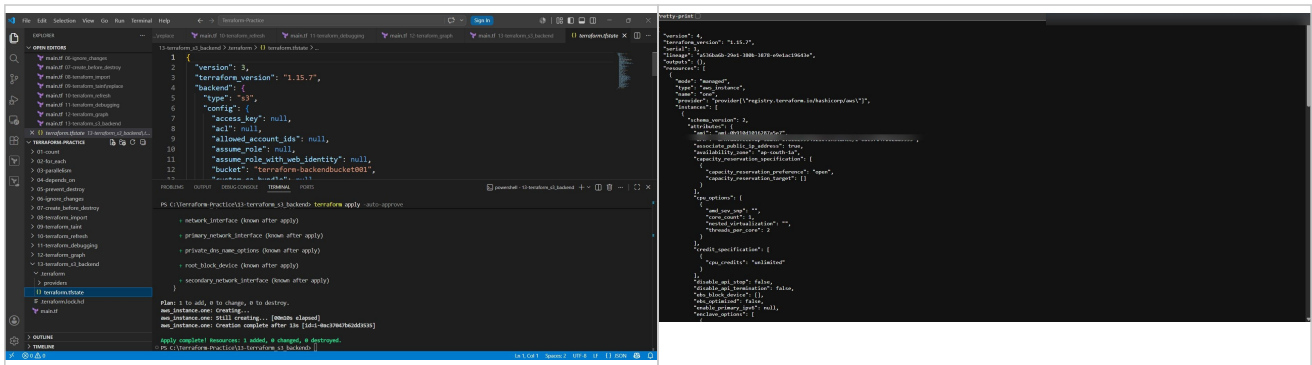


Left: the Terraform code defining a VPC, subnet, and instance. Right: the generated dependency graph, visually showing the flow from `aws_vpc.one` to `aws_subnet.two` to `aws_instance.three`.

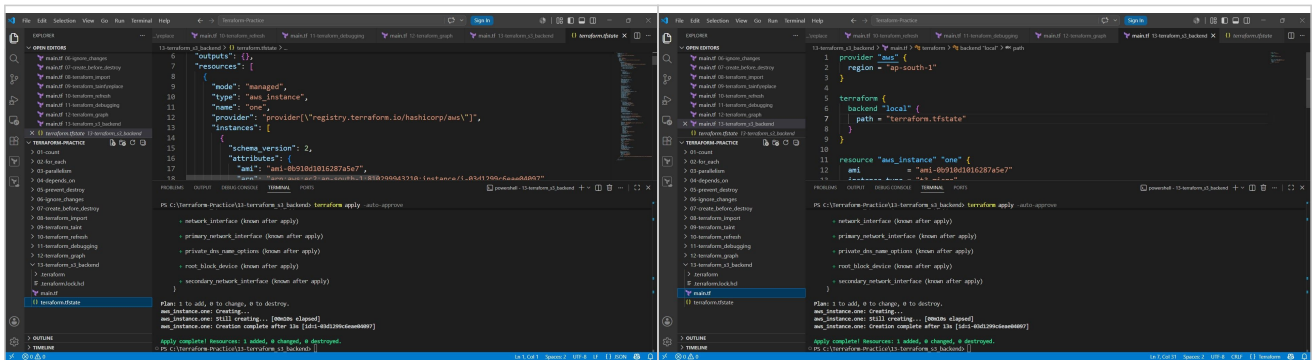
Remote State With an S3 Backend

What is Terraform state, and why does it matter? Every time Terraform runs, it keeps a record (the “state file”) of exactly what infrastructure it created and how everything is configured. By default this file is saved locally on your own computer — which is a problem if you’re working in a team, since everyone needs to see the same, up-to-date state.

The fix: a *remote backend* stores that state file somewhere shared instead — in this case, an S3 bucket. Now anyone on the team running Terraform pulls the same shared state, avoiding conflicts.



Left: the backend block in the Terraform code, pointing to an S3 bucket (“terraform-backendbucket001”) to store the state file. Right: the actual terraform.tfstate file, opened directly from that S3 bucket — the browser’s address bar and an account ID inside the file have been blurred, since a state file can reveal sensitive infrastructure details and shouldn’t be shared or left open.



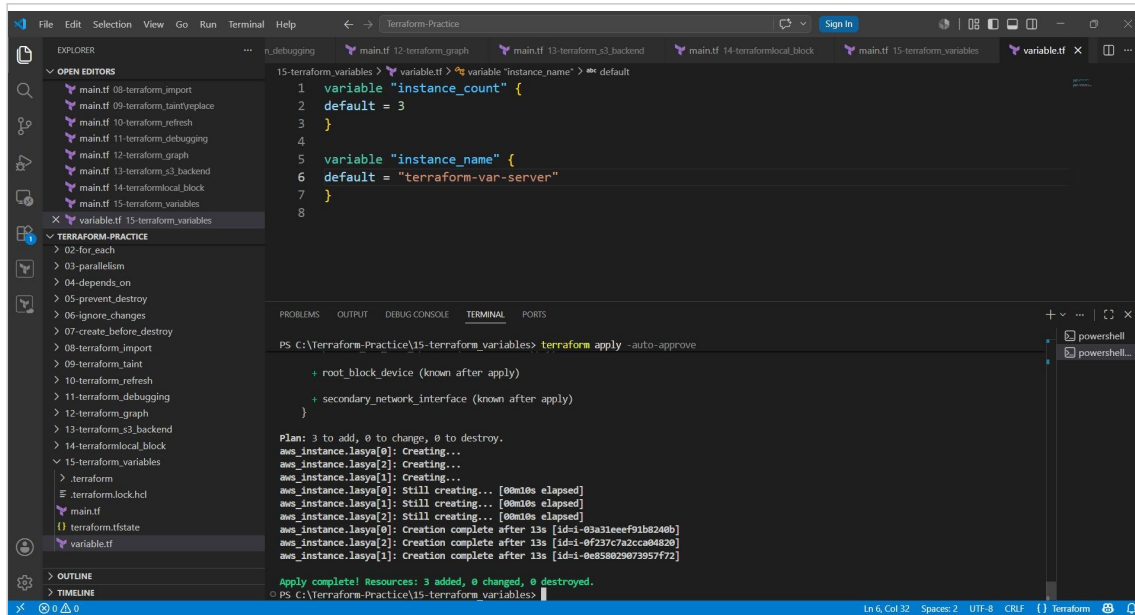
Both panels show the same state file and backend configuration from different angles — confirming Terraform is reading and writing its state to the shared S3 location instead of a local file.

Good practice: a Terraform state file can contain sensitive values (resource IDs, IP addresses, and sometimes even secrets), so the S3 bucket storing it should always have public access blocked and encryption enabled.

Writing Reusable Configuration

14 local blocks — naming a value once, reusing it everywhere

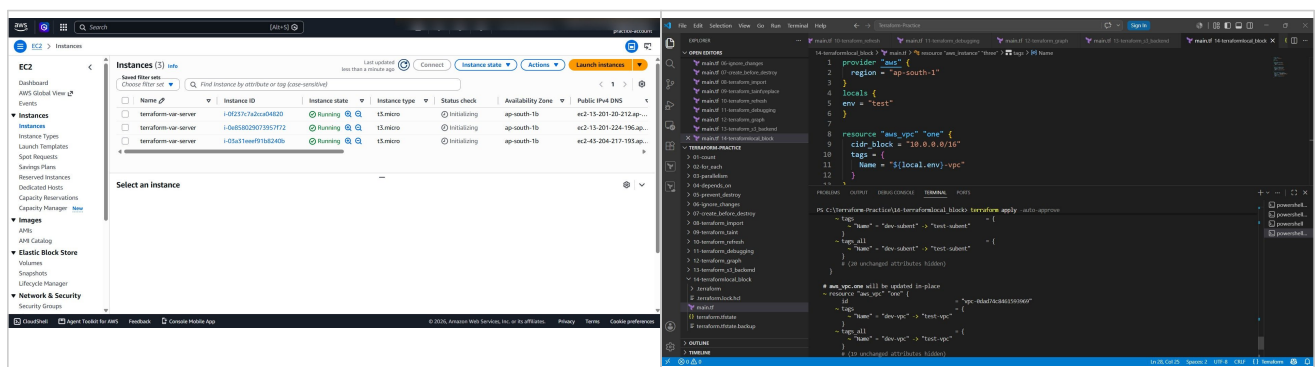
A locals block lets you define a value once (like a naming prefix or an environment name) and reuse it throughout your configuration by referring to local.<name>. This avoids repeating the same value everywhere and makes it easy to change it in one place later.



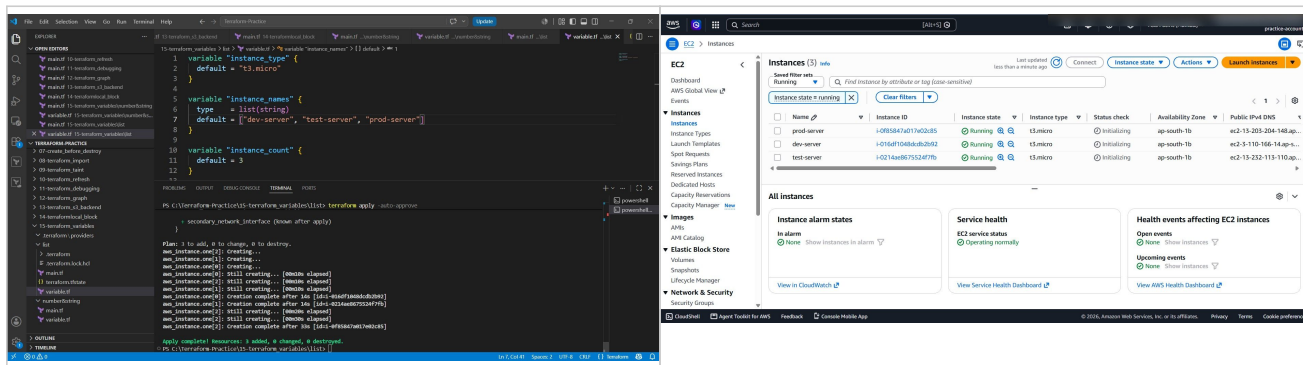
A locals block defining an environment value, used inside a resource's tags so every resource created gets a consistent, centrally-defined name.

15 variables — making configuration flexible

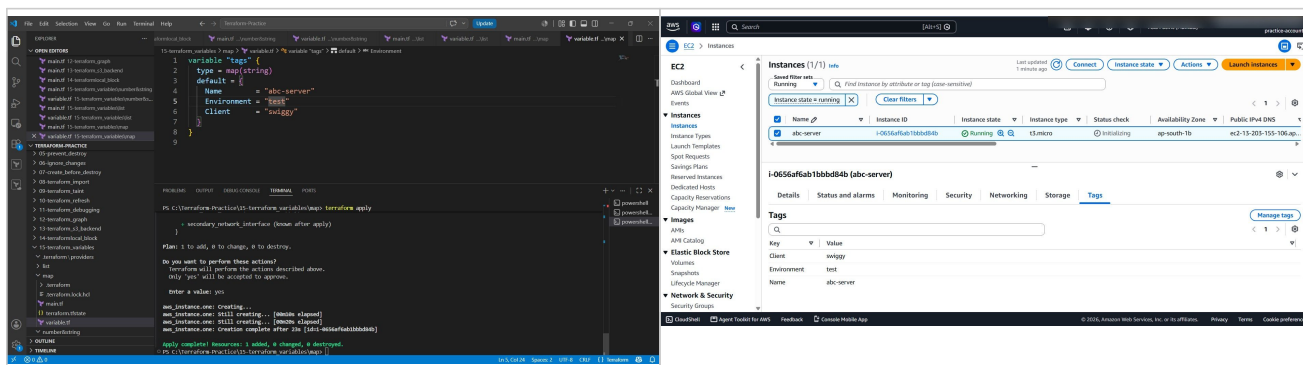
Instead of hardcoding values like the number of servers or their names directly into your code, variable blocks let you define inputs that can be changed without touching the main configuration — similar to function parameters in programming. This makes the same Terraform code reusable across different situations.



Left: variables defined for instance count and name, with default values. Right: the resulting servers created using those variable values, visible in the EC2 console.



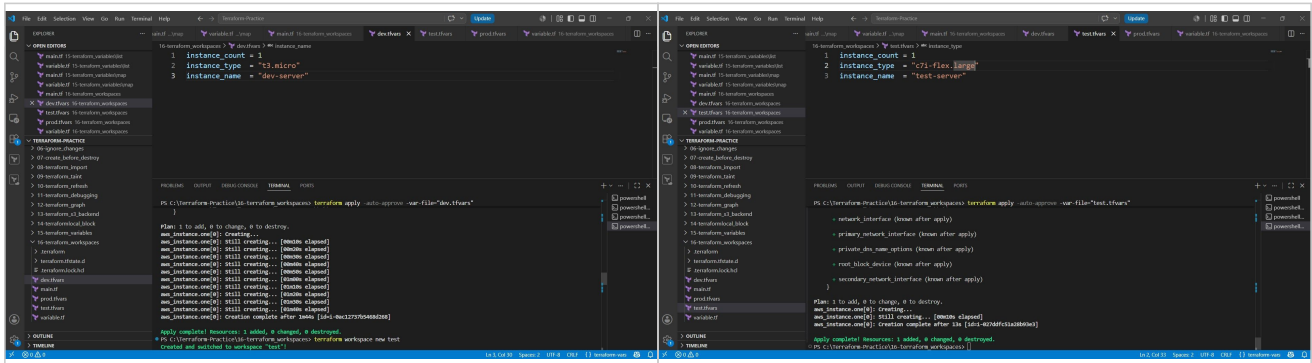
Left: a variable holding a list of instance names, so multiple differently-named servers can be created from one flexible block. Right: those servers now running, each correctly named.



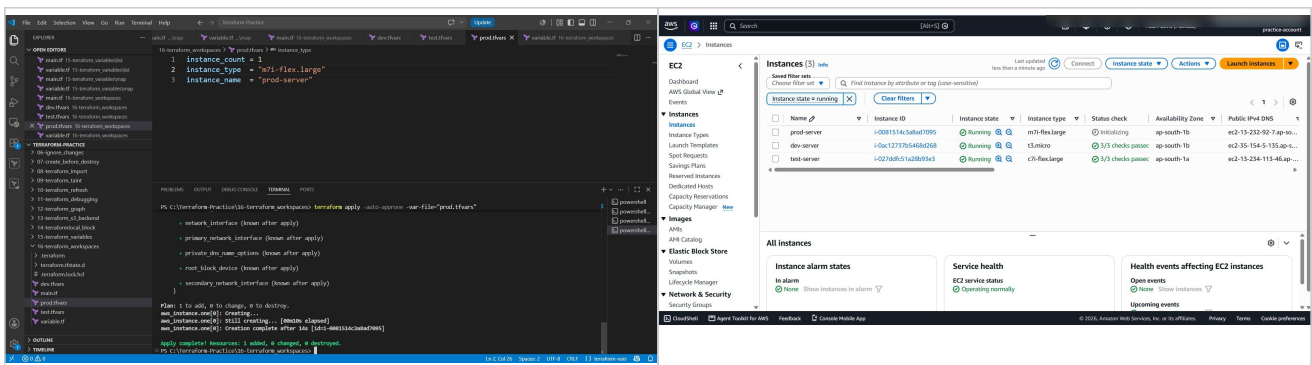
Left: a variable being used to tag resources with environment and client information. Right: those tags visible directly on the running instance in the console, confirming the variable values were applied correctly.

Terraform Workspaces

The problem: most projects need separate dev, test, and production environments — but writing entirely separate Terraform code for each one is wasteful. **Workspaces** solve this by letting the same configuration be applied multiple times, once per environment, each with its own separate state file. Switching workspaces (with `terraform workspace select`) changes which environment's state Terraform is currently working with, without changing the code itself.



Left: applying the configuration inside a “dev” workspace, using the `terraform.workspace` value to automatically name the server accordingly. Right: the resulting dev-environment server running in the console.

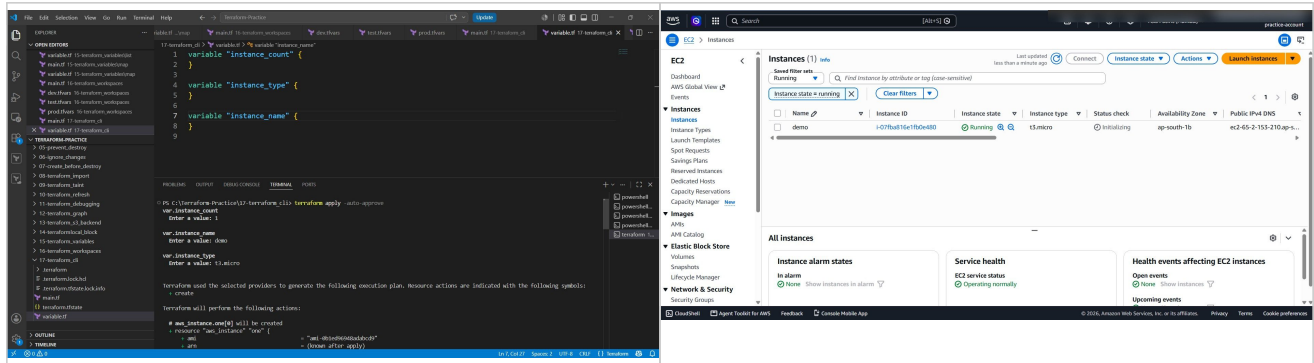


Left: the exact same configuration applied again, this time inside a “prod” workspace. Right: a separate, independent production server created — all instances across every workspace now visible together in the console, each correctly labeled.

The Terraform CLI & Dynamic Blocks

17 The Terraform CLI — the commands behind everything

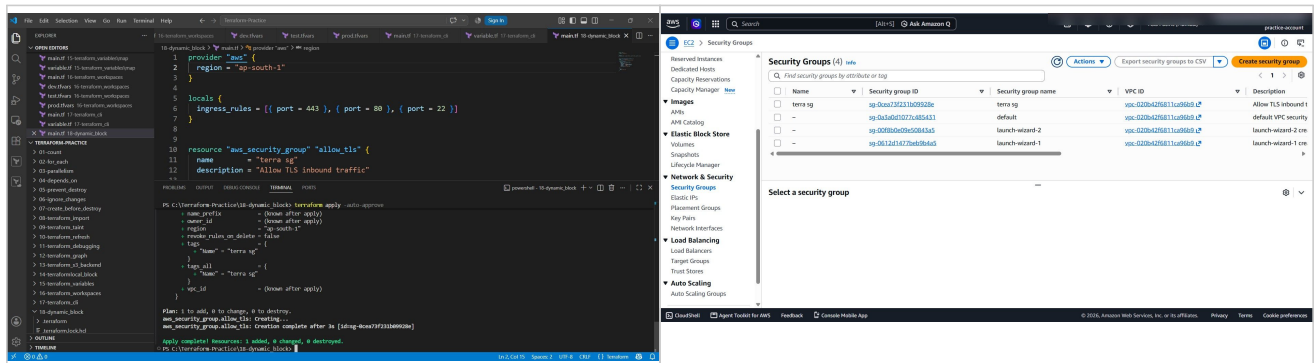
Every example in this document runs through a small set of core Terraform CLI commands: `terraform init` (set up the working directory and download providers), `terraform plan` (preview what will change), `terraform apply` (actually make the changes), and `terraform destroy` (tear resources back down). Understanding these four commands is really the foundation of using Terraform day to day.



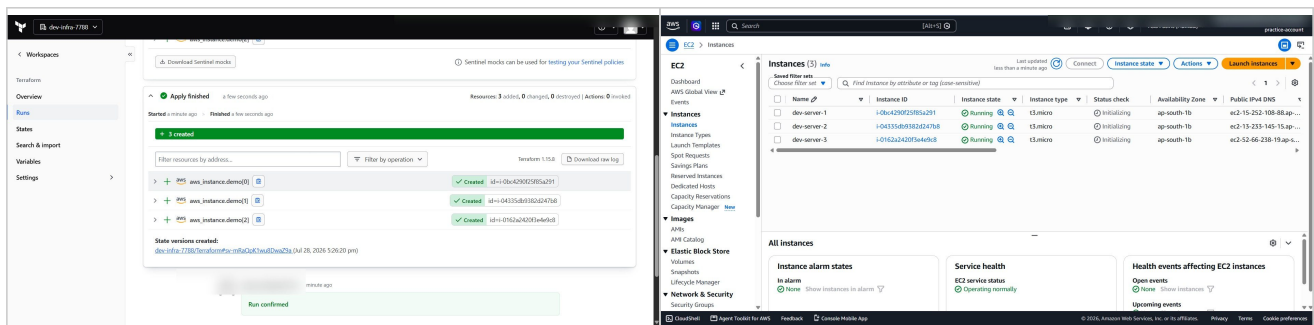
Left: variables being defined for a resource, ready to be applied through the standard CLI workflow. Right: the resulting instance confirmed running in the console.

18 dynamic blocks — generating repeated nested blocks in code

Some AWS resources (like security groups) need multiple similar nested blocks — for example, one “ingress rule” block per port you want to open. A dynamic block lets you generate all of those nested blocks automatically from a list, instead of writing each one out by hand.



Left: a dynamic block generating multiple ingress rules (for ports 443, 80, and 22) from a single list defined in locals. Right: the resulting security group in the AWS console, showing all the generated rules.



Left: the same configuration applied through Terraform Cloud, showing a confirmed run and the resources it created — the account username has been blurred. Right: the matching instances now visible in the EC2 console.

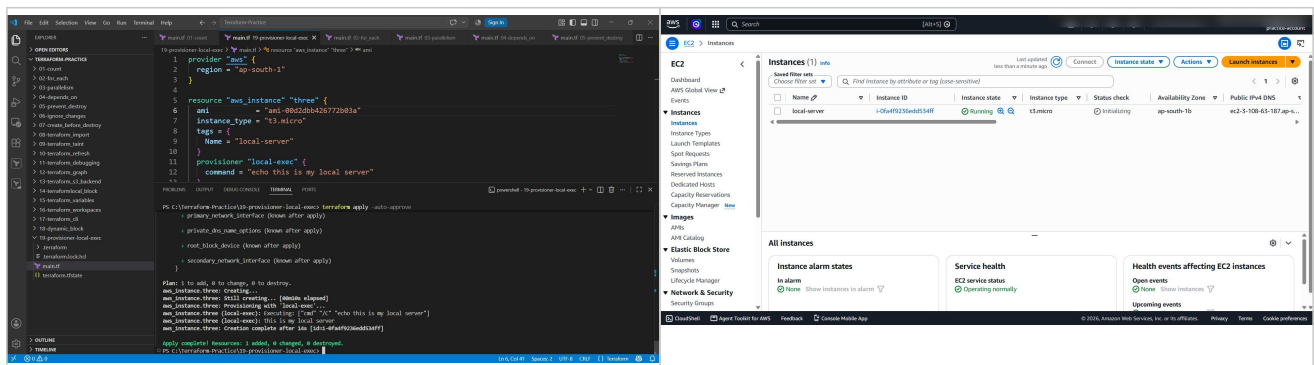
Provisioners

What is a provisioner? Terraform's main job is creating infrastructure, not configuring what runs inside it.

Provisioners are an escape hatch for running a command either on your own machine or on the newly created server itself, right after it's built — useful for quick setup tasks, though dedicated configuration tools are usually preferred for anything more complex.

19 local-exec — running a command on your own machine

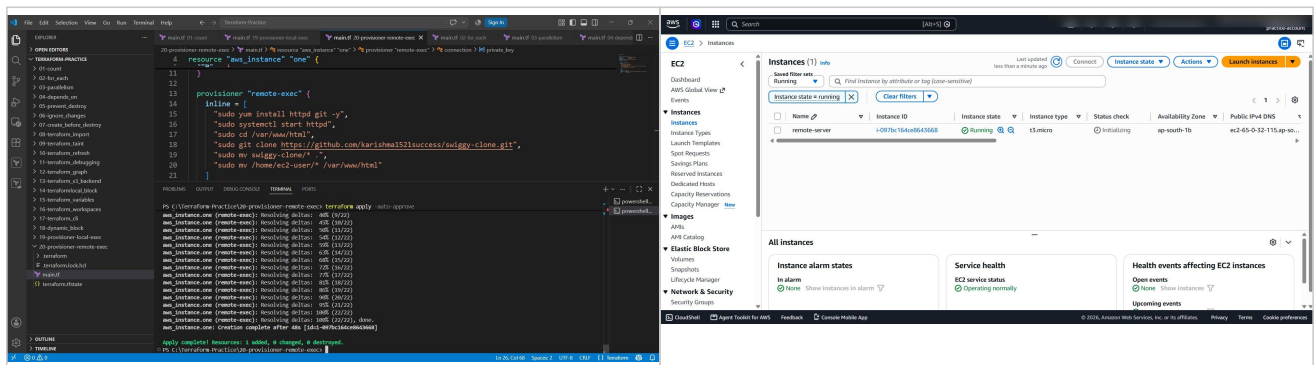
A local-exec provisioner runs a command on the computer running Terraform (not on the new server) immediately after that resource is created — for example, printing a confirmation message or triggering a local script.



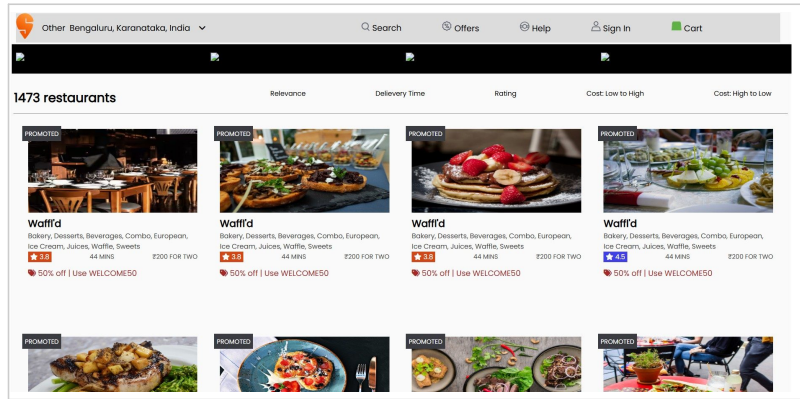
Left: a local-exec provisioner set to echo a message once the instance is created, with the terminal confirming it ran. Right: the resulting “local-server” instance running in EC2.

20 remote-exec — running commands on the new server itself

A remote-exec provisioner connects to the newly created server (typically over SSH) and runs commands directly on it — for example, installing software like Git right after the server boots up, without needing to log in manually afterward.



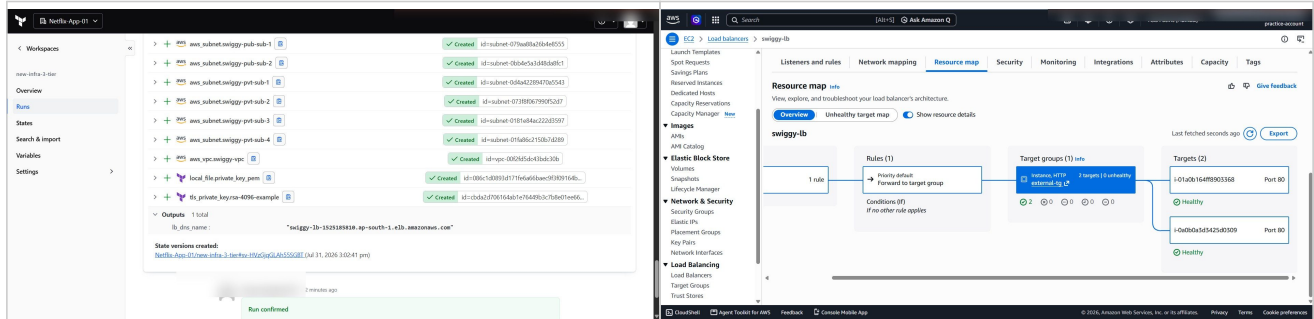
Left: a remote-exec provisioner installing Git and cloning a repository directly on the new server as soon as it's ready. Right: the “remote-server” instance running successfully in the console.



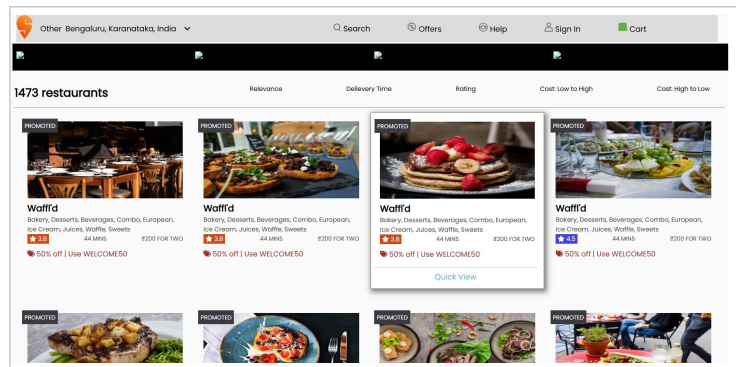
The application that was automatically set up on the server through the remote-exec provisioner, now live and reachable.

Final Project — 3-Tier AWS Architecture

This project brings together everything from the topics above into one real deployment: a **3-tier architecture** built entirely with Terraform, including a custom VPC, public and private subnets, a load balancer, and EC2 instances — managed through Terraform Cloud so the whole team can see the same run history and state.



Left: the Terraform Cloud run confirming every piece of infrastructure was created successfully — subnets, VPC, and even a generated private key — with the account username blurred. Right: the resulting load balancer's resource map in the AWS Console, showing traffic flowing from the load balancer's rules through to two healthy target servers.



The final live application, served through the load balancer that Terraform built — proof that the entire chain, from infrastructure code to a working, publicly-accessible app, was successfully automated end to end.

End of notes. All screenshots in this document were cropped and redacted for privacy before publishing.